

Homework #2

February 10, 2021

Chun-Yuan (Scott) Chiu
chunyuac@andrew.cmu.edu

1.

(a)

Read in the data and train a default gradient boosting. Cross-validation suggests to use 14 trees for optimal results.

```
[11]: import pandas as pd
import xgboost as xgb
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

X = pd.read_csv('marketing.csv')

for col in X.columns:
    if col not in ['age', 'balance']:
        X[col] = X[col].astype("category")

X = pd.get_dummies(X).drop('y_no', axis='columns').rename(columns={'y_yes': 'y'})
y = X.pop('y')

X_tmp, X_final, y_tmp, y_final = train_test_split(X, y, test_size=0.20,
    random_state=1)
X_train, X_test, y_train, y_test = train_test_split(X_tmp, y_tmp, test_size=0.
    random_state=10)

train_dm = xgb.DMatrix(X_train.values, y_train.values)
test_dm = xgb.DMatrix(X_test.values, y_test.values)
final_dm = xgb.DMatrix(X_final.values, y_final.values)

param = {'learning_rate': 0.3,
        'max_depth': 6,
```

```

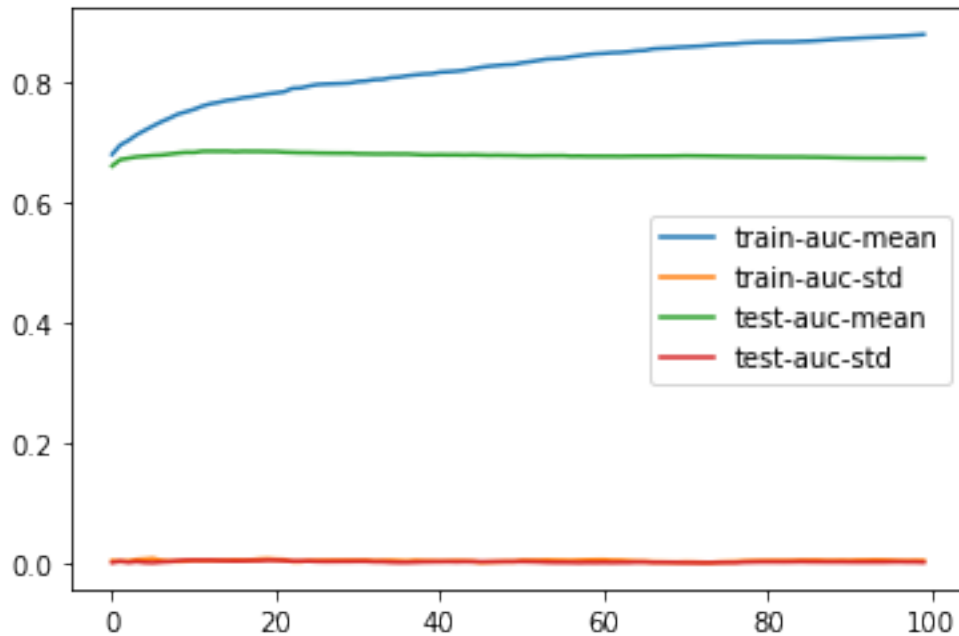
    'objective': 'binary:logistic',
    'seed': 0,
    'eval_metric': 'auc',
}

```

```

[4]: cvres = xgb.cv(param, train_dm, 100)
cvres.plot()
plt.show()
cvres['test-auc-mean'].argmax()

```



[4]: 14

We also train a random forest classifier. Both models are cached so that we do not have to retrain later.

```

[2]: from sklearn.ensemble import RandomForestClassifier
import pickle, lzma

default_bst = xgb.train(param, train_dm, 14)
rf = RandomForestClassifier(n_estimators=500, oob_score=True).fit(X_train,
    ↪ y_train)

for i, fitted in enumerate([default_bst, rf]):
    with lzma.open(f'clf{i}.xz', 'wb') as f:
        f.write(pickle.dumps(fitted))

```

```
[2]: import lzma, pickle

with lzma.open('clf0.xz', 'rb') as f:
    default_bst = pickle.loads(f.read())

with lzma.open('clf1.xz', 'rb') as f:
    rf = pickle.loads(f.read())
```

Below is the ROC curves, the misclassification rate, and best 1000. Overall the default boosting already performs better than the random forest.

```
[9]: from sklearn.metrics import accuracy_score, roc_curve
import matplotlib.pyplot as plt

predict_proba_bst = default_bst.predict(test_dm)
predict_proba_rf = rf.predict_proba(X_test)[:, 1]

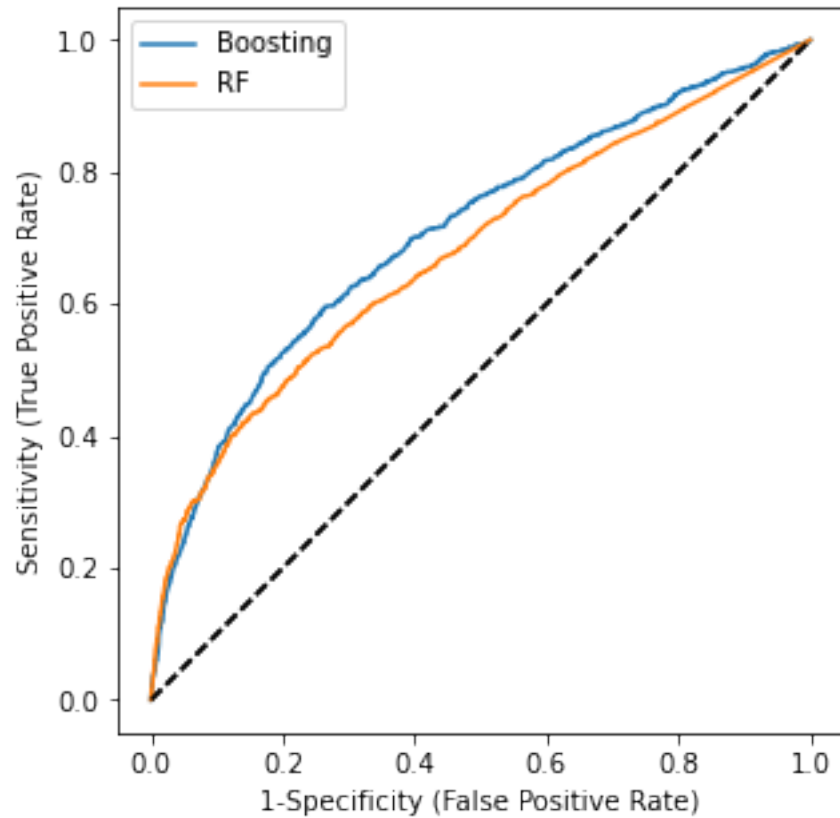
predict_proba_map = {
    'Boosting': predict_proba_bst,
    'RF': predict_proba_rf}

fig, ax = plt.subplots(1, 1, figsize=(5, 5))

for name, predict_proba in predict_proba_map.items():
    fpr, tpr, _ = roc_curve(y_test, predict_proba)
    ax.plot(fpr, tpr, label=name)
    ax.plot([0, 1], [0, 1], 'k--')
    ax.set( aspect=1,
            xlabel='1-Specificity (False Positive Rate)',
            ylabel='Sensitivity (True Positive Rate)')

fig.suptitle('ROC Curves', fontsize=14)
plt.legend()
plt.show()
```

ROC Curves



```
[21]: from pandas import DataFrame

threshold = 0.5

res = {key: [] for key, _ in predict_proba_map.items()}

for name, predict_proba in predict_proba_map.items():
    bestIdxSet = predict_proba.argsort()[-1000:]
    y_select = y_test.values[bestIdxSet]
    res[name].append(y_select.sum()/len(y_select))

    y_pred = predict_proba > threshold
    res[name].append(1 - accuracy_score(y_test, y_pred))

DataFrame(res, index=['Best 1000', 'Misclassification'])
```

```
[21]:
```

	Boosting	RF
Best 1000	0.349000	0.347000

Misclassification 0.117452 0.126189

(b)

We started with a parameter set defined as `param1` below. There is a way to specify custom eval metric in `xgboost.cv`. We will be using the prediction error rate in the best 1000 candidates by the model as the eval metric, but we report the opposite, that is 1 minus the prediction error rate, which we name `best1000`.

The optimal `best1000` 0.312 is achieved with 45 trees.

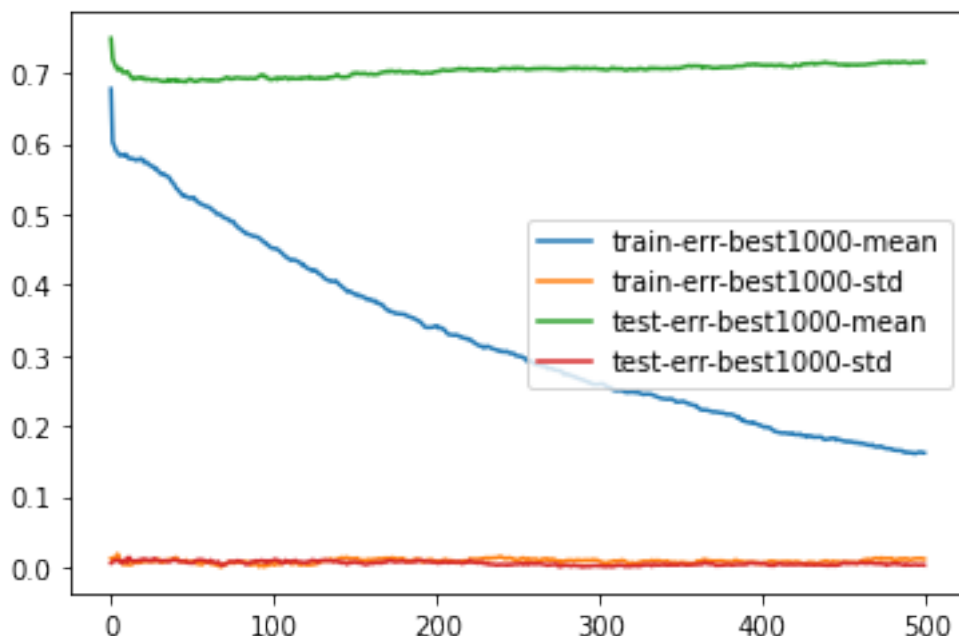
```
[57]: %%time

param1 = {'learning_rate': 0.1,
          'max_depth': 6,
          'objective': 'binary:logistic',
          'gamma': 0,
          'subsample': 0.8,
          'colsample_bytree': 0.8,
          'scale_pos_weight': 1,
          'disable_default_eval_metric': 1,
          'tree_method': 'gpu_hist',
          'seed': 0,
          }

def errBest1000(preds, dtrain):
    labels = dtrain.get_label()
    bestIdxSet = preds.argsort()[-1000:]
    y_select = labels[bestIdxSet]
    return 'err-best1000', 1 - y_select.sum()/1000

cvres = xgb.cv(param1, train_dm, 500, feval=errBest1000)
cvres.plot()
plt.show()

print('optimal n_estimators: ', cvres['test-err-best1000-mean'].argmin())
print('optimal test-best1000: ', 1-cvres['test-err-best1000-mean'].min())
print()
```



```
optimal n_estimators: 45
optimal test-best1000: 0.31199999999999994
```

```
CPU times: user 15.7 s, sys: 6.86 s, total: 22.5 s
Wall time: 22.5 s
```

For GridSearchCV there is a way to specify the scoring (the opposite of eval metric) but it can only be a function of the binary prediction results, not the predicted score. Thus compute `best1000` is not possible. We instead use PPV as a proxy, optimizing the probability of a candidate actually saying yes if he/she is predicted to say yes.

Tuning `max_depth` and `min_child_weight`, the optimal `best1000` improves to 0.316 with 57 trees.

```
[59]: %%time
from sklearn.model_selection import GridSearchCV
from xgboost.sklearn import XGBClassifier
from sklearn.metrics import make_scorer, confusion_matrix

param_grid1 = {
    'max_depth': [4, 6, 8, 10],
    'min_child_weight': [1, 2, 3, 4, 5],
}

def ppv(y_true, y_pred):
    # positive predictive value
    tn, fp, fn, tp = confusion_matrix(y_true.values, y_pred).ravel()
    return tp/(tp + fp)
```

```

ppv_score = make_scorer(ppv)

grid_search = GridSearchCV(estimator=XGBClassifier(n_estimators=45, **param1),
                           param_grid=param_grid1,
                           scoring=ppv_score, cv=5).fit(X_train, y_train)

idx = grid_search.cv_results_['rank_test_score'].argmin()
print('optimal: ', grid_search.cv_results_['params'][idx])
print('best score: ', grid_search.cv_results_['mean_test_score'][idx])
print()

```

/usr/local/lib/python3.6/dist-packages/ipykernel_launcher.py:13: RuntimeWarning:
invalid value encountered in long_scalars

```
del sys.path[0]
```

optimal: {'max_depth': 10, 'min_child_weight': 1}

best score: 0.44539729238423265

CPU times: user 1min 3s, sys: 36.1 s, total: 1min 39s

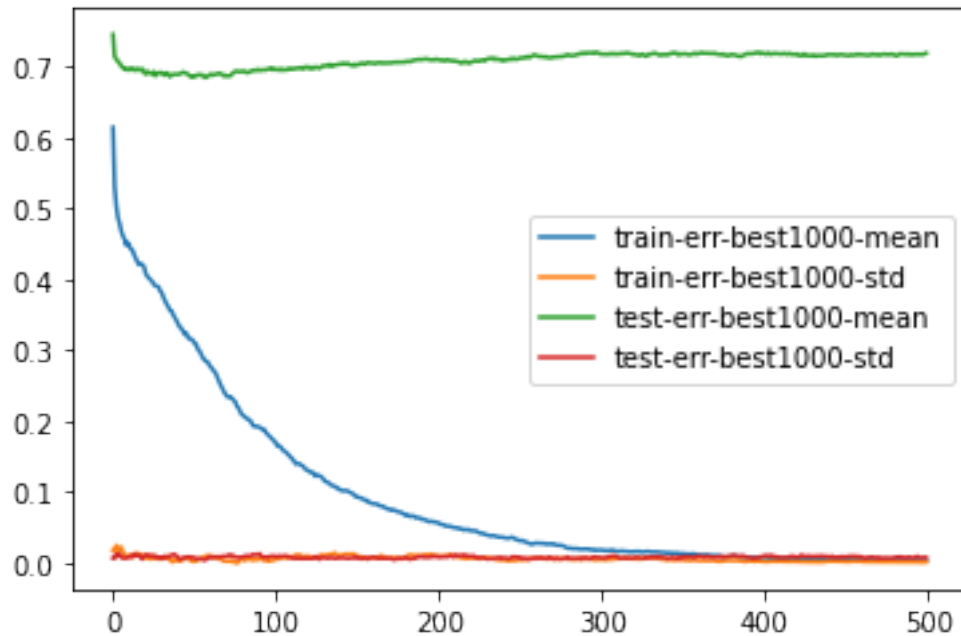
Wall time: 1min 39s

```

[60]: %%time
param1.update(grid_search.cv_results_['params'][idx])

cvres = xgb.cv(param1, train_dm, 500, feval=errBest1000)
cvres.plot()
plt.show()
print('optimal n_estimators: ', cvres['test-err-best1000-mean'].argmin())
print('optimal test-best1000: ', 1-cvres['test-err-best1000-mean'].min())
print()

```



```
optimal n_estimators: 57
optimal test-best1000: 0.31599999999999995
```

```
CPU times: user 36 s, sys: 14.6 s, total: 50.6 s
Wall time: 50.6 s
```

The optimal `max_depth` was a corner case so we tune `max_depth` again alone. As a result the optimal is still 10; using deeper tree will not improve best1000.

```
[61]: %%time

param_grid1 = {
    'max_depth': [8, 10, 12, 14],
}

grid_search = GridSearchCV(estimator=XGBClassifier(n_estimators=57, **param1),
                           param_grid=param_grid1,
                           scoring=ppv_score, cv=5).fit(X_train, y_train)

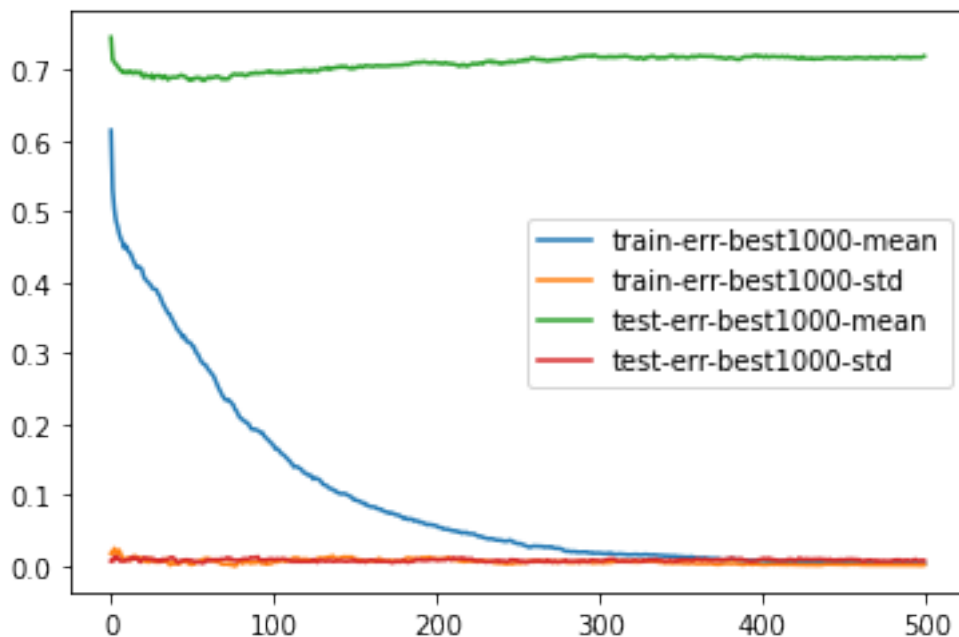
idx = grid_search.cv_results_['rank_test_score'].argmin()
print('optimal: ', grid_search.cv_results_['params'][idx])
print('best score: ', grid_search.cv_results_['mean_test_score'][idx])
print()
```

```
optimal: {'max_depth': 10}
best score: 0.485117898046261
```


CPU times: user 39.6 s, sys: 17.8 s, total: 57.5 s
Wall time: 57.4 s

```
[62]: %%time
param1.update(grid_search.cv_results_['params'][idx])

cvres = xgb.cv(param1, train_dm, 500, feval=errBest1000)
cvres.plot()
plt.show()
print('optimal n_estimators: ', cvres['test-err-best1000-mean'].argmin())
print('optimal test-best1000: ', 1-cvres['test-err-best1000-mean'].min())
print()
```



optimal n_estimators: 57
optimal test-best1000: 0.31599999999999995

CPU times: user 35.8 s, sys: 14.9 s, total: 50.7 s
Wall time: 50.7 s

We then try to tune `gamma` and `reg_alpha` but the result suggests that the default value 0 is the best for both.

```
[64]: %%time
param_grid1 = {
    'gamma': [0, 0.2, 0.4, 0.6],
}
```

```

grid_search = GridSearchCV(estimator=XGBClassifier(n_estimators=57, **param1),
                           param_grid=param_grid1,
                           scoring=ppv_score, cv=5).fit(X_train, y_train)

idx = grid_search.cv_results_['rank_test_score'].argmin()
print('optimal: ', grid_search.cv_results_['params'][idx])
print('best score: ', grid_search.cv_results_['mean_test_score'][idx])
print()

```

```

optimal:  {'gamma': 0}
best score:  0.485117898046261

```

```

CPU times: user 31.3 s, sys: 14.8 s, total: 46.1 s
Wall time: 46.1 s

```

```

[69]: %%time
param_grid1 = {
    'reg_alpha': [0, 1e-5, 1e-2, 0.1, 1, 100]
}

grid_search = GridSearchCV(estimator=XGBClassifier(n_estimators=57, **param1),
                           param_grid=param_grid1,
                           scoring=ppv_score, cv=5).fit(X_train, y_train)

idx = grid_search.cv_results_['rank_test_score'].argmin()
print('optimal: ', grid_search.cv_results_['params'][idx])
print('best score: ', grid_search.cv_results_['mean_test_score'][idx])
print()

```

```

/usr/local/lib/python3.6/dist-packages/ipykernel_launcher.py:13: RuntimeWarning:
invalid value encountered in long_scalars
    del sys.path[0]

```

```

optimal:  {'reg_alpha': 0}
best score:  0.46939683184038683

```

```

CPU times: user 1min 21s, sys: 37.3 s, total: 1min 59s
Wall time: 1min 58s

```

Tuning the tree parameters `subsample` and `colsample_bytree`, the result suggest we had a lucky guess from the beginning. The optimal `best1000` remains 0.316 with 57 trees.

```

[67]: %%time
param_grid1 = {
    'subsample': [0.6, 0.7, 0.8, 0.9],
    'colsample_bytree': [0.6, 0.7, 0.8, 0.9]
}

```

```

grid_search = GridSearchCV(estimator=XGBClassifier(n_estimators=57, **param1),
                           param_grid=param_grid1,
                           scoring=ppv_score, cv=5).fit(X_train, y_train)

idx = grid_search.cv_results_['rank_test_score'].argmin()
print('optimal: ', grid_search.cv_results_['params'][idx])
print('best score: ', grid_search.cv_results_['mean_test_score'][idx])
print()

```

optimal: {'colsample_bytree': 0.8, 'subsample': 0.8}
best score: 0.485117898046261

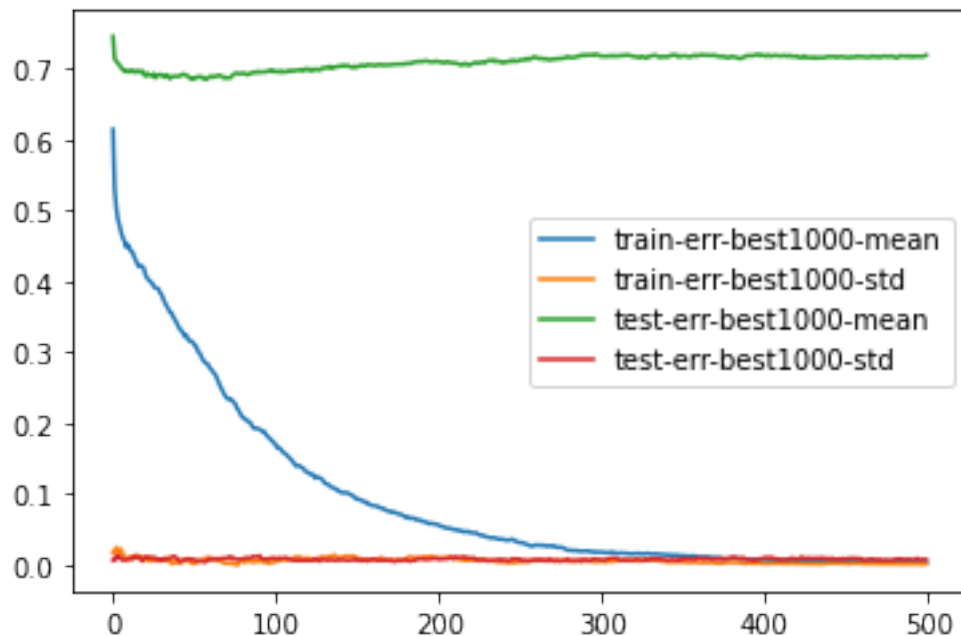
CPU times: user 1min 56s, sys: 53.3 s, total: 2min 49s
Wall time: 2min 49s

```

[68]: %%time
param1.update(grid_search.cv_results_['params'][idx])

cvres = xgb.cv(param1, train_dm, 500, feval=errBest1000)
cvres.plot()
plt.show()
print('optimal n_estimators: ', cvres['test-err-best1000-mean'].argmin())
print('optimal test-best1000: ', 1-cvres['test-err-best1000-mean'].min())
print()

```



optimal n_estimators: 57

optimal test-best1000: 0.31599999999999995

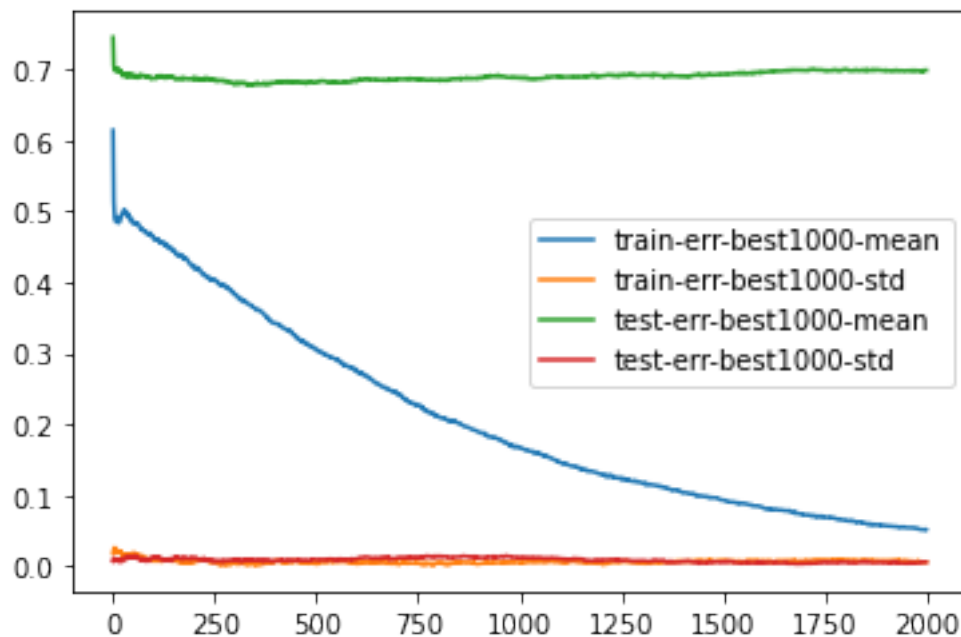
CPU times: user 36.1 s, sys: 14.7 s, total: 50.8 s

Wall time: 50.8 s

Finally we lower `learning_rate` to allow more trees to contribute in the prediction. The optimal `best1000` improves to 0.3223 with 330 trees.

```
[70]: %%time
param1.update({'learning_rate': 0.01})

cvres = xgb.cv(param1, train_dm, 2000, feval=errBest1000)
cvres.plot()
plt.show()
print('optimal n_estimators: ', cvres['test-err-best1000-mean'].argmin())
print('optimal test-best1000: ', 1-cvres['test-err-best1000-mean'].min())
print()
```



optimal n_estimators: 330

optimal test-best1000: 0.32233333333333336

CPU times: user 2min 20s, sys: 56.4 s, total: 3min 17s

Wall time: 3min 17s

Below are the fine tuned parameter set.

```
[71]: param1
```

```
[71]: {'colsample_bytree': 0.8,
      'disable_default_eval_metric': 1,
      'gamma': 0,
      'learning_rate': 0.01,
      'max_depth': 10,
      'min_child_weight': 1,
      'objective': 'binary:logistic',
      'scale_pos_weight': 1,
      'seed': 0,
      'subsample': 0.8,
      'tree_method': 'gpu_hist'}
```

We retrain and save cache the model.

```
[ ]: tuned_bst = xgb.train({'colsample_bytree': 0.8,
                           'disable_default_eval_metric': 1,
                           'gamma': 0,
                           'learning_rate': 0.01,
                           'max_depth': 10,
                           'min_child_weight': 1,
                           'objective': 'binary:logistic',
                           'scale_pos_weight': 1,
                           'seed': 0,
                           'subsample': 0.8}, train_dm, 330)

with lzma.open(f'clf2.xz', 'wb') as f:
    f.write(pickle.dumps(tuned_bst))
```

```
[3]: import lzma, pickle

with lzma.open('clf0.xz', 'rb') as f:
    default_bst = pickle.loads(f.read())

with lzma.open('clf1.xz', 'rb') as f:
    rf = pickle.loads(f.read())

with lzma.open('clf2.xz', 'rb') as f:
    tuned_bst = pickle.loads(f.read())
```

Below are the ROC curves, misclassification rate and reported best1000. With fine tuned parameters, best1000 has significant improvement.

```
[8]: from sklearn.metrics import accuracy_score, roc_curve
import matplotlib.pyplot as plt

predict_proba_tuned_bst = tuned_bst.predict(test_dm)
predict_proba_bst = default_bst.predict(test_dm)
predict_proba_rf = rf.predict_proba(X_test)[: , 1]
```

```

predict_proba_map = {
    'Tuned Boosting': predict_proba_tuned_bst,
    'Default Boosting': predict_proba_bst,
    'RF': predict_proba_rf}

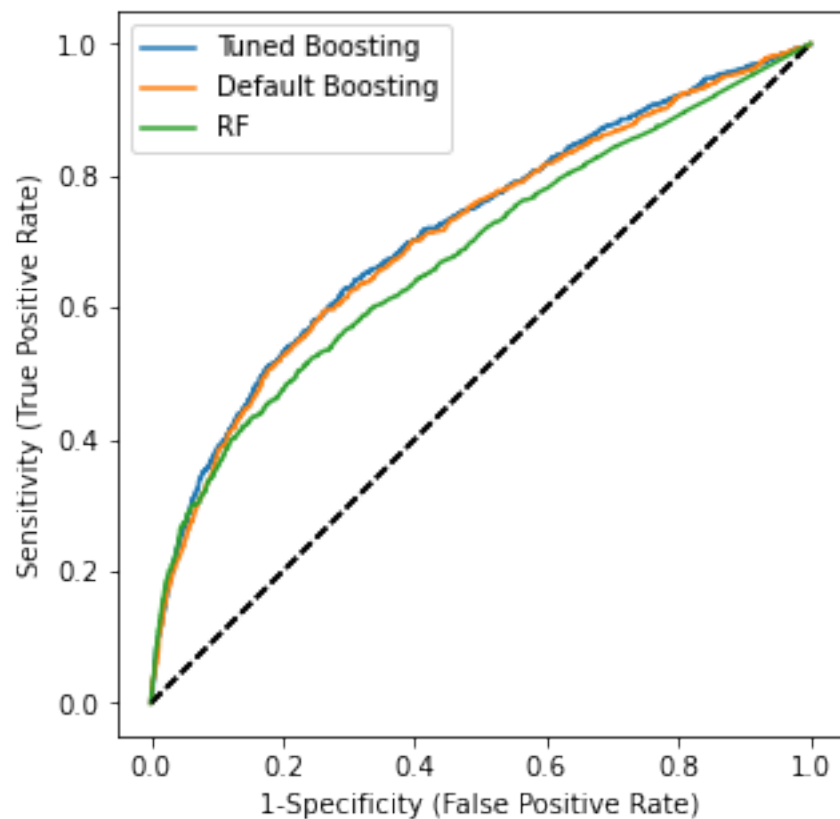
fig, ax = plt.subplots(1, 1, figsize=(5, 5))

for name, predict_proba in predict_proba_map.items():
    fpr, tpr, _ = roc_curve(y_test, predict_proba)
    ax.plot(fpr, tpr, label=name)
    ax.plot([0, 1], [0, 1], 'k--')
    ax.set( aspect=1,
            xlabel='1-Specificity (False Positive Rate)',
            ylabel='Sensitivity (True Positive Rate)')

fig.suptitle('ROC Curves', fontsize=14)
plt.legend()
plt.show()

```

ROC Curves



```
[9]: from pandas import DataFrame

threshold = 0.5

res = {key: [] for key, _ in predict_proba_map.items()}

for name, predict_proba in predict_proba_map.items():
    bestIdxSet = predict_proba.argsort()[-1000:]
    y_select = y_test.values[bestIdxSet]
    res[name].append(y_select.sum()/len(y_select))

    y_pred = predict_proba > threshold
    res[name].append(1 - accuracy_score(y_test, y_pred))

DataFrame(res, index=['Best 1000', 'Misclassification'])
```

```
[9]:
```

	Tuned Boosting	Default Boosting	RF
Best 1000	0.377000	0.349000	0.347000
Misclassification	0.118005	0.117452	0.126189

(c)

On the final test set, however, despite the ROC curve of the fine tuned boosting still looks better than other models, the `best1000` reduces to the same level as random forest.

```
[13]: from sklearn.metrics import accuracy_score, roc_curve
import matplotlib.pyplot as plt

predict_proba_tuned_bst = tuned_bst.predict(final_dm)
predict_proba_bst = default_bst.predict(final_dm)
predict_proba_rf = rf.predict_proba(X_final)[: , 1]

predict_proba_map = {
    'Tuned Boosting': predict_proba_tuned_bst,
    'Default Boosting': predict_proba_bst,
    'RF': predict_proba_rf}

fig, ax = plt.subplots(1, 1, figsize=(5, 5))

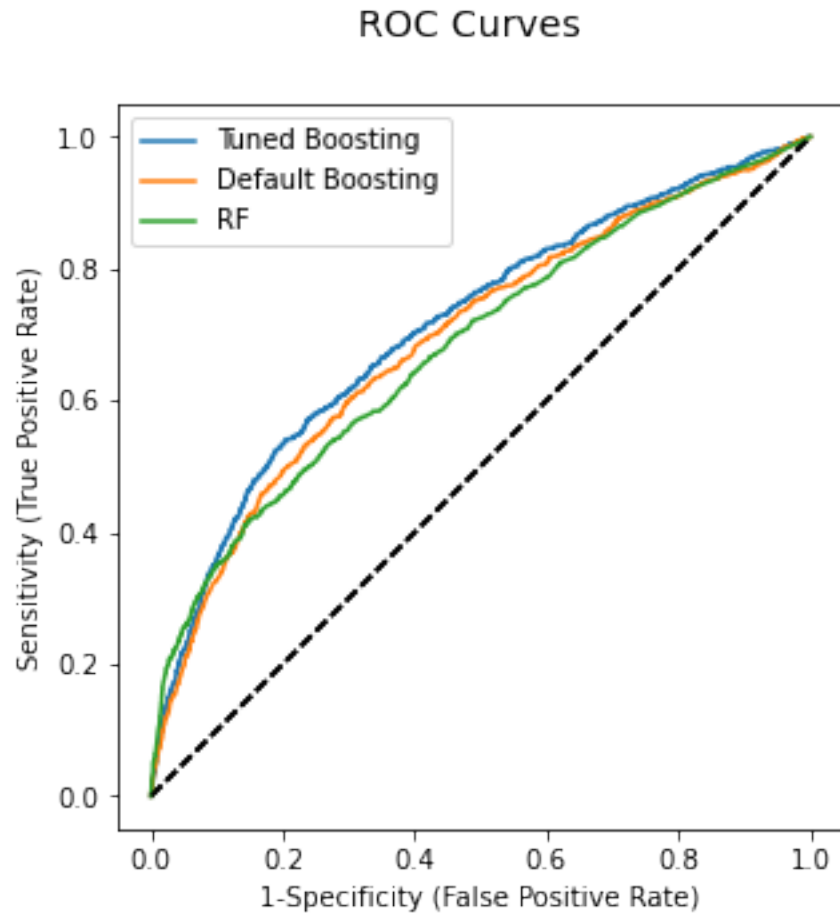
for name, predict_proba in predict_proba_map.items():
    fpr, tpr, _ = roc_curve(y_final, predict_proba)
    ax.plot(fpr, tpr, label=name)
    ax.plot([0, 1], [0, 1], 'k--')
    ax.set( aspect=1,
            xlabel='1-Specificity (False Positive Rate)',
```

```

        ylabel='Sensitivity (True Positive Rate)')

fig.suptitle('ROC Curves', fontsize=14)
plt.legend()
plt.show()

```



```

[16]: from pandas import DataFrame

threshold = 0.5

res = {key: [] for key, _ in predict_proba_map.items()}

for name, predict_proba in predict_proba_map.items():
    bestIdxSet = predict_proba.argsort()[-1000:]
    y_select = y_final.values[bestIdxSet]
    res[name].append(y_select.sum()/len(y_select))

y_pred = predict_proba > threshold

```



```
res[name].append(1 - accuracy_score(y_final, y_pred))

DataFrame(res, index=['Best 1000', 'Misclassification'])
```

```
[16]:
```

	Tuned Boosting	Default Boosting	RF
Best 1000	0.338000	0.320000	0.338000
Misclassification	0.116665	0.115559	0.126838

2.

(a)

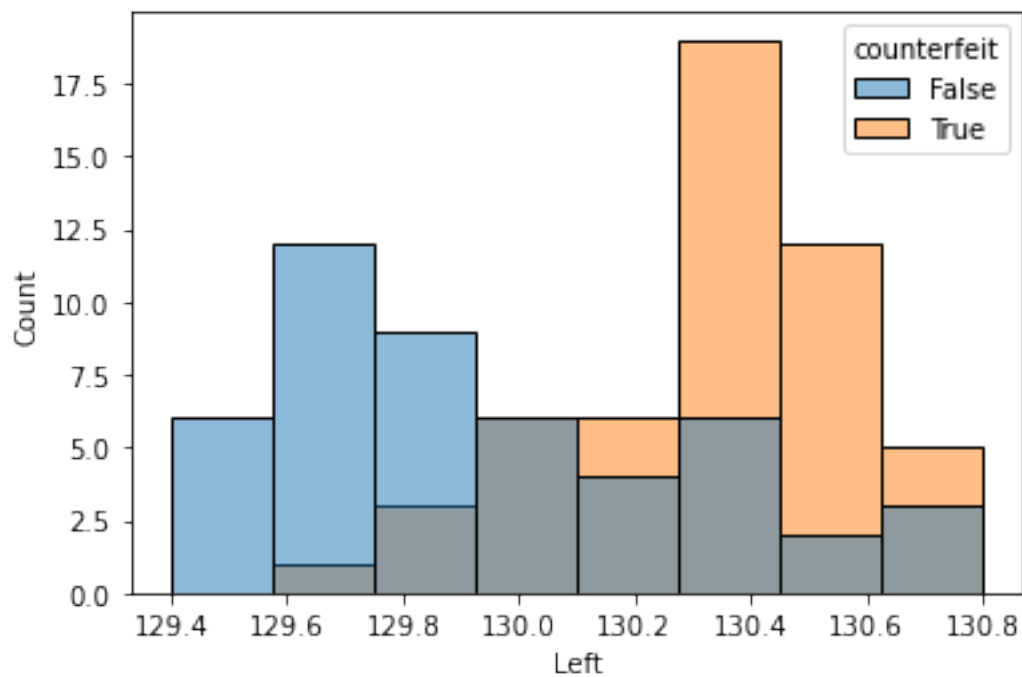
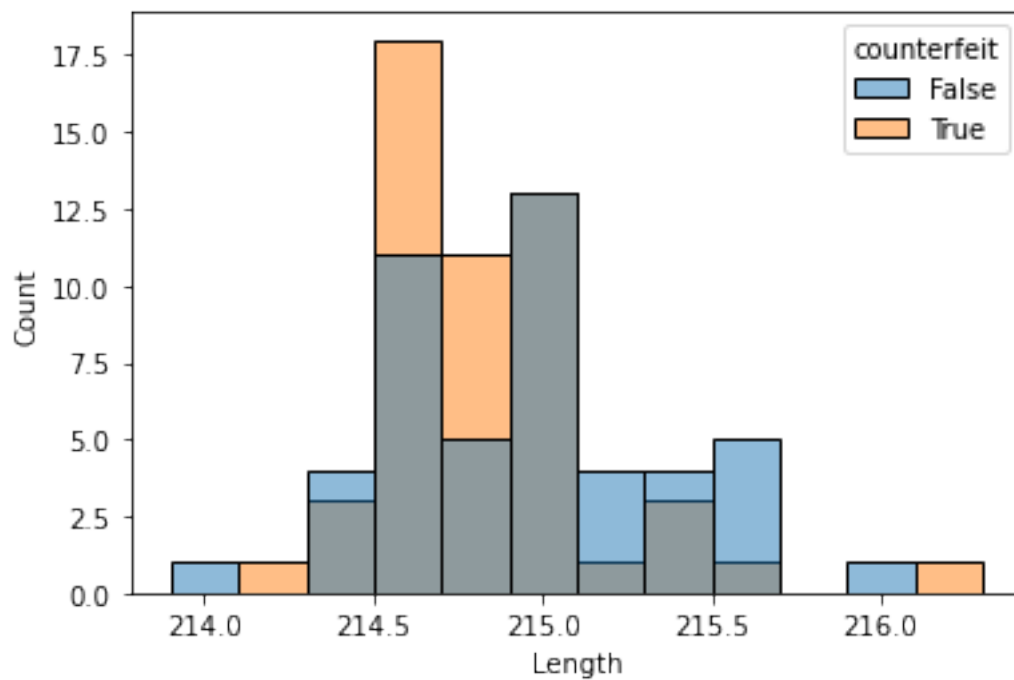
Below are the plots. We see no feature perfectly separating the prediction results.

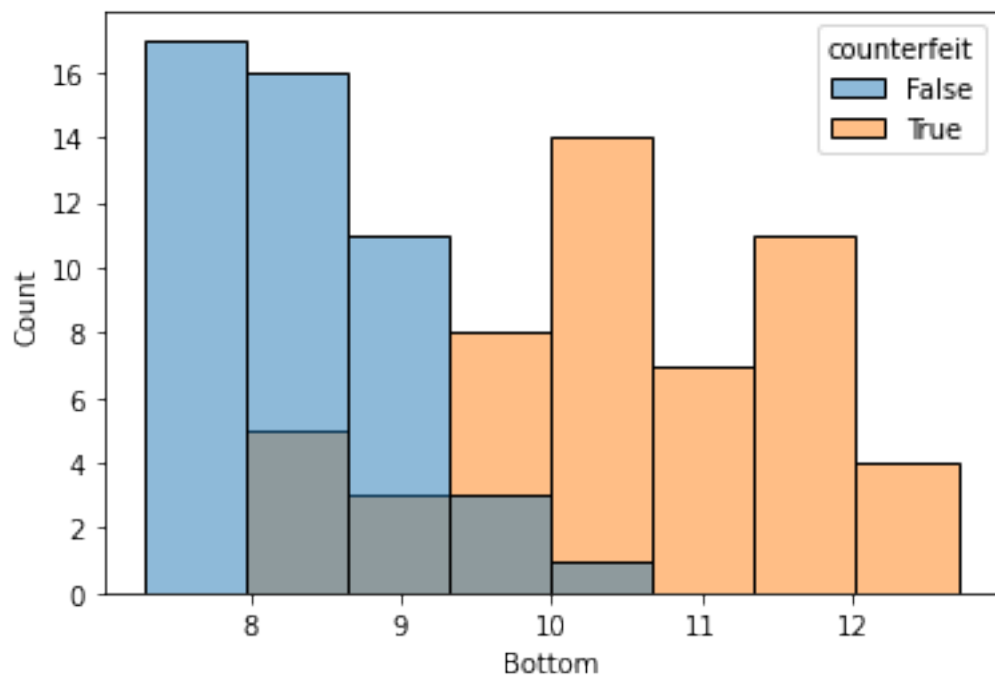
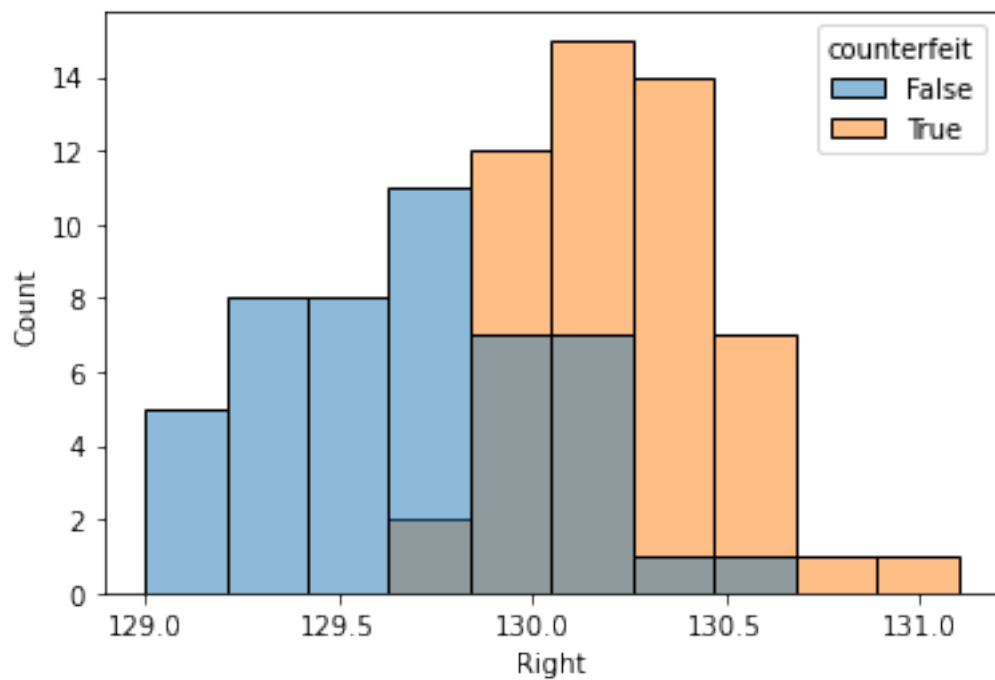
```
[1]: from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

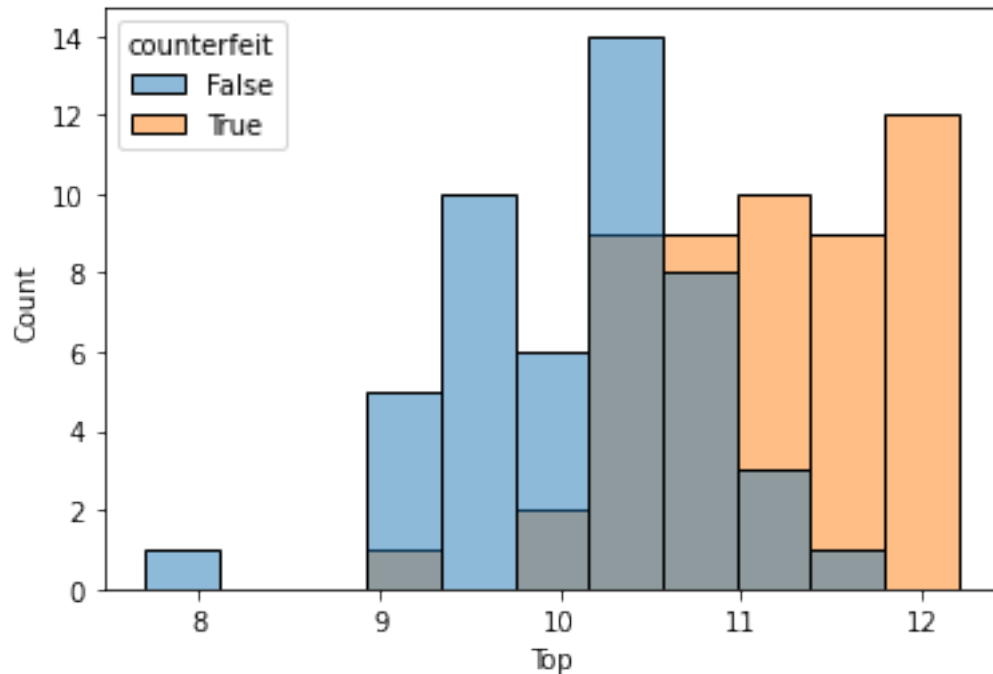
X = pd.read_csv('banknote_measurements.csv', index_col=0)
y = (X.pop('counterfeit') == 'counterfeit')

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5,
→random_state=1)

for col in X_test.columns:
    sns.histplot(data=X_test, x=col, hue=y_test)
    plt.show()
```







(b)

We fit an LDA model and report confusion matrix. The misclassification rate is 7%.

```
[2]: from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
      from sklearn.metrics import confusion_matrix
      from pandas import DataFrame

      clf = LinearDiscriminantAnalysis().fit(X_train, y_train)
      tn, fp, fn, tp = confusion_matrix(y_test, clf.predict(X_test)).ravel()

      DataFrame([[tp, fp], [fn, tn]], columns=['True Y=1', 'True Y=0'], index=['Guess Y=1', 'Guess Y=0'])
```

```
[2]:      True Y=1  True Y=0
      Guess Y=1      46      1
      Guess Y=0       6     47
```

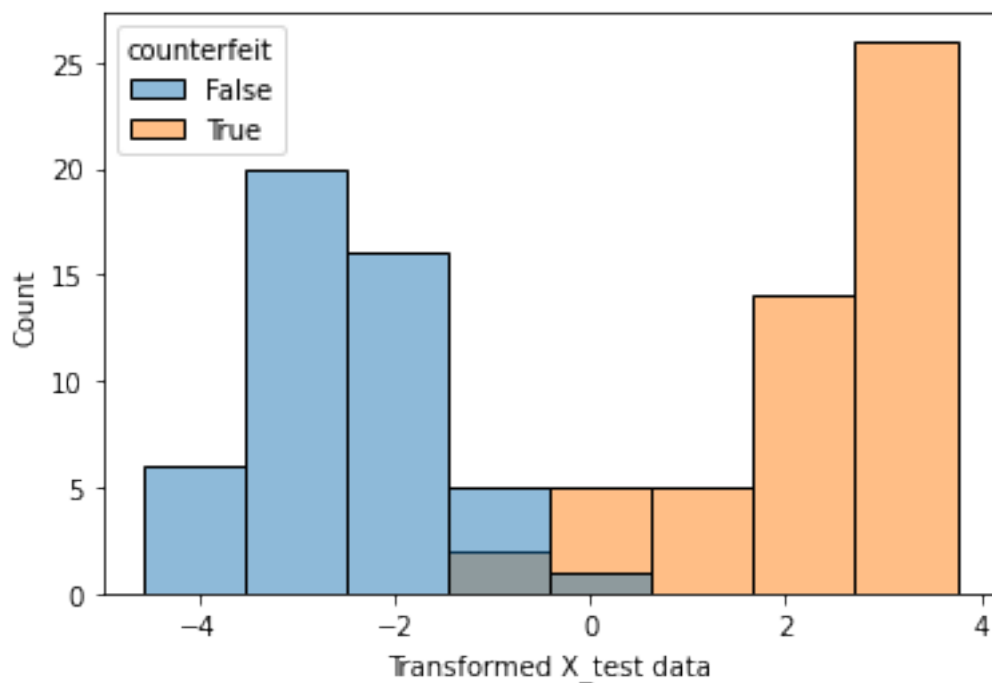
```
[3]: (fn + fp)/(tn + fp + fn + tp)
```

```
[3]: 0.07
```

(c)

There is a much more clear separation in the histogram of the transformed feature, compared to the original ones.

```
[52]: ax = sns.histplot(x=clf.transform(X_test).flatten(), hue=y_test)
ax.set(xlabel='Transformed X_test data')
plt.show()
```



3.

(a)

```
[27]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

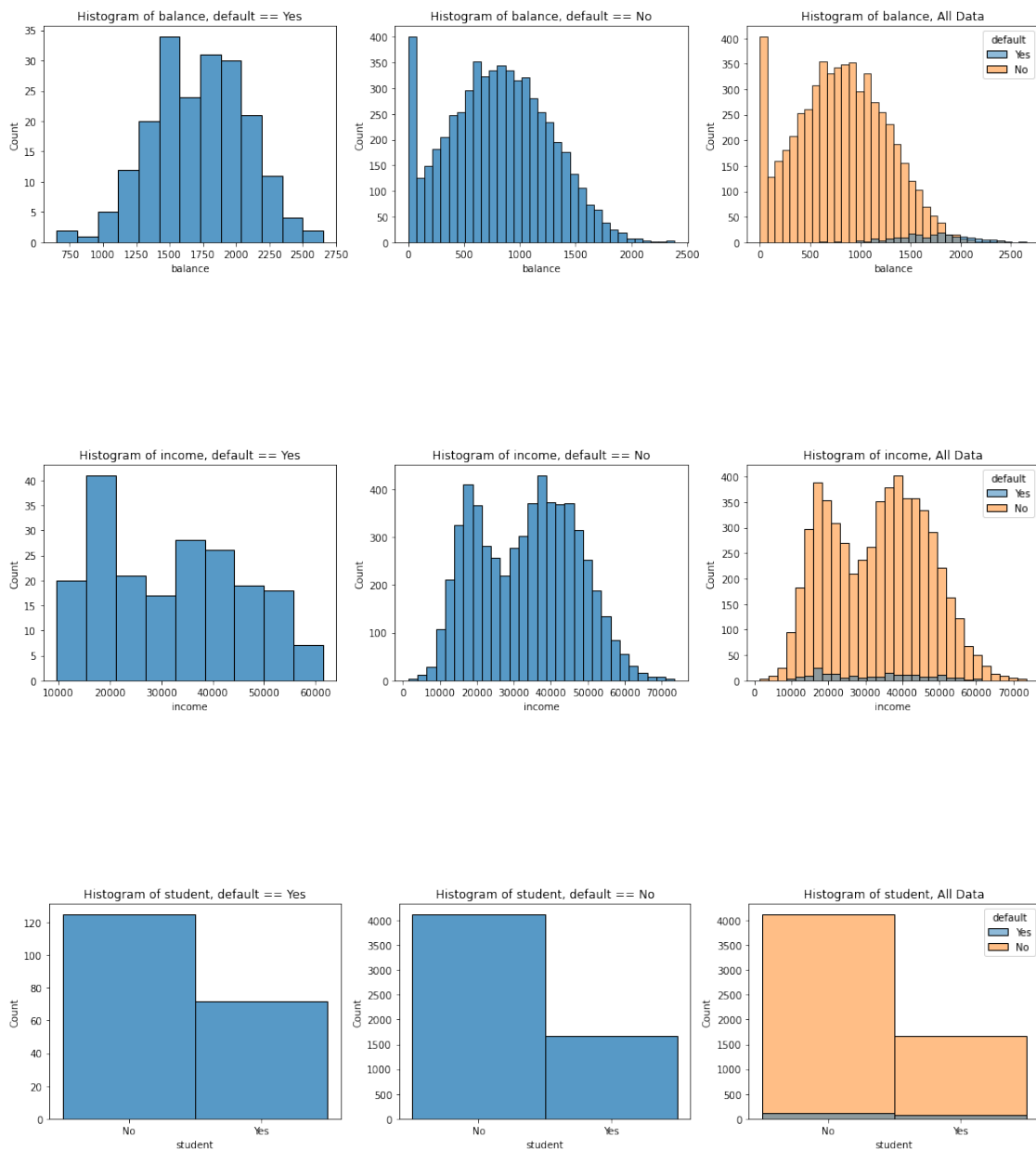
X_train = pd.read_csv('default_train.csv')
y_train = X_train.pop('default')

X_test = pd.read_csv('default_test.csv')
y_test = X_test.pop('default')
```

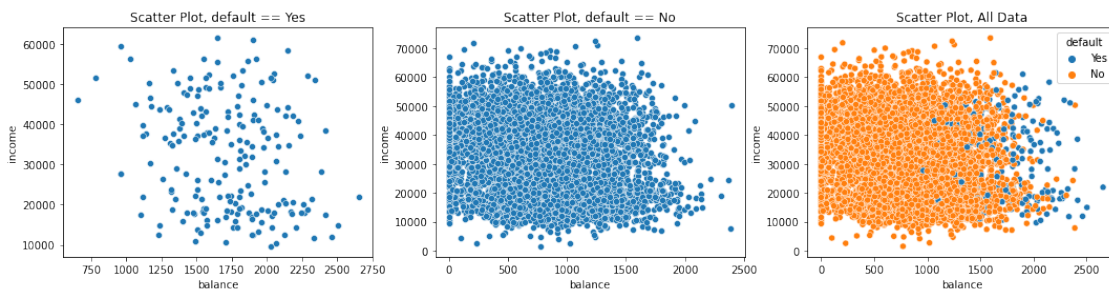
```

for col in ['balance', 'income', 'student']:
    fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(18, 4))
    sns.histplot(x=col, data=X_train[y_train=='Yes'], ax=ax1)
    sns.histplot(x=col, data=X_train[y_train=='No'], ax=ax2)
    sns.histplot(x=col, data=X_train, hue=y_train, ax=ax3)
    ax1.set(title=f'Histogram of {col}, default == Yes')
    ax2.set(title=f'Histogram of {col}, default == No')
    ax3.set(title=f'Histogram of {col}, All Data')
plt.show()
print()

```



```
[28]: fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(18, 4))
sns.scatterplot(x='balance', y='income', ax=ax1, data=X_train[y_train=='Yes'])
sns.scatterplot(x='balance', y='income', ax=ax2, data=X_train[y_train=='No'])
sns.scatterplot(x='balance', y='income', ax=ax3, data=X_train, hue=y_train)
ax1.set(title='Scatter Plot, default == Yes')
ax2.set(title='Scatter Plot, default == No')
ax3.set(title='Scatter Plot, All Data')
plt.show()
```

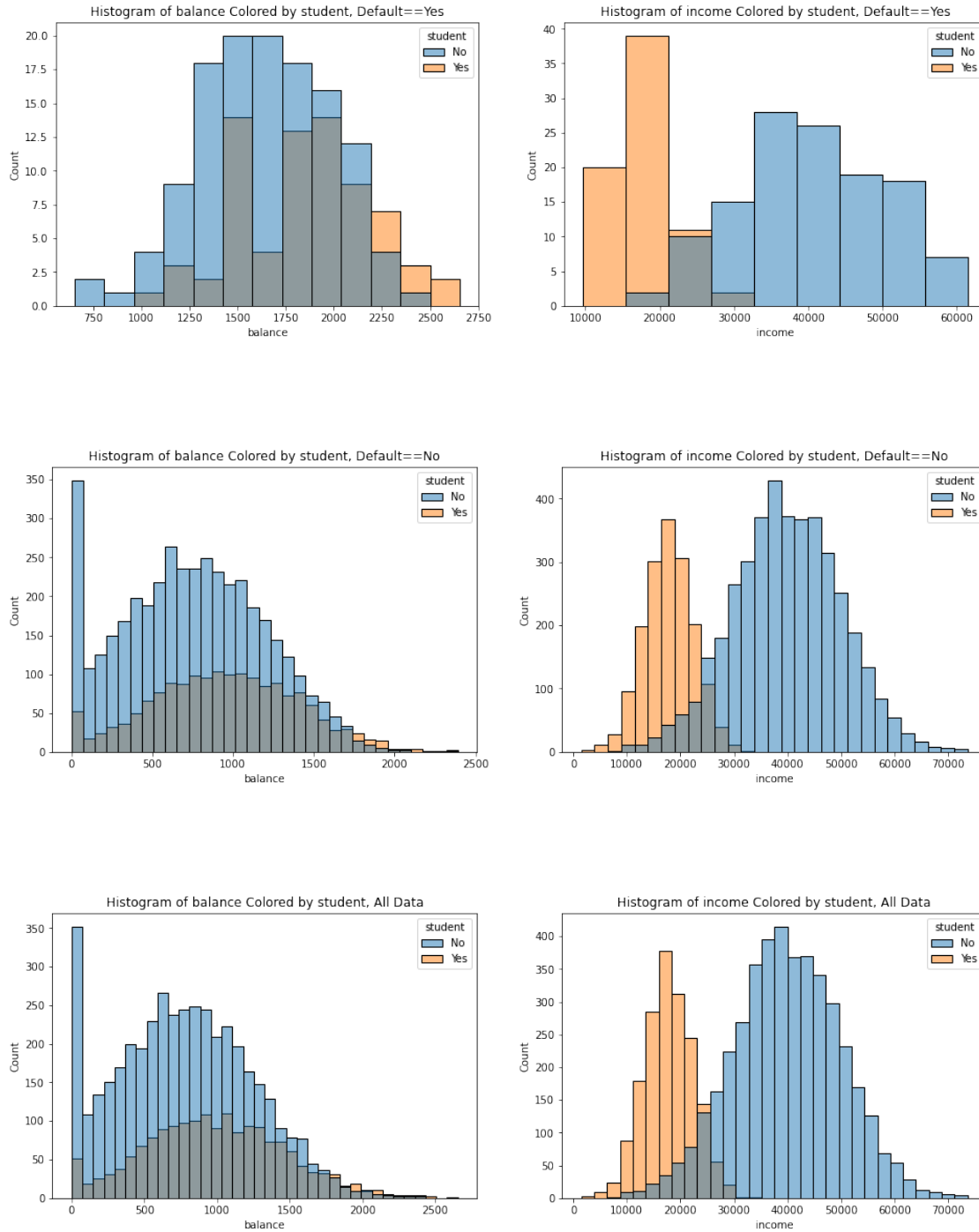


Given the below histogram, the distributions of income and student conditioned on Default is clearly dependent. Students like me have much lower income on average which makes sense. This contradicts the naive Bayes assumption.

```
[29]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
sns.histplot(x='balance', data=X_train[y_train=='Yes'], hue='student', ax=ax1)
sns.histplot(x='income', data=X_train[y_train=='Yes'], hue='student', ax=ax2)
ax1.set(title='Histogram of balance Colored by student, Default==Yes')
ax2.set(title='Histogram of income Colored by student, Default==Yes')
plt.show()
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
sns.histplot(x='balance', data=X_train[y_train=='No'], hue='student', ax=ax1)
sns.histplot(x='income', data=X_train[y_train=='No'], hue='student', ax=ax2)
ax1.set(title='Histogram of balance Colored by student, Default==No')
ax2.set(title='Histogram of income Colored by student, Default==No')
plt.show()
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
sns.histplot(x='balance', data=X_train, hue='student', ax=ax1)
sns.histplot(x='income', data=X_train, hue='student', ax=ax2)
ax1.set(title='Histogram of balance Colored by student, All Data')
ax2.set(title='Histogram of income Colored by student, All Data')
plt.show()
```



(b)

For normal model, we use the `fit` method provided in `scipy.stats.norm`, which according to the documentation reports the MLE estimates. For discrete distributions like `bernoulli`, `fit` is not

implemented yet (although there is an Issue #11948 on GitHub requesting it) so we hard code the parameter. The fitted parameters are reported as below.

```
[30]: from scipy.stats import norm, bernoulli
from pandas import DataFrame

dist_norm = {}
for feature in ['balance', 'income']:
    for default in ['Yes', 'No']:
        params = norm.fit(X_train[y_train==default][feature])
        dist_norm[feature + ' | ' + default] = norm(*params)

dist_norm['student | Yes'] =
    ↪bernoulli(sum(X_train[y_train=='Yes']['student']=='Yes')/
    ↪len(X_train[y_train=='Yes']))
dist_norm['student | No'] =
    ↪bernoulli(sum(X_train[y_train=='No']['student']=='Yes')/
    ↪len(X_train[y_train=='No']))

df = DataFrame([(k, None, *d.stats()) for k, d in dist_norm.items()],
    ↪columns=['Feature|Default', 'Distribution Type', 'Mean', 'Variance'])
df['Distribution Type'] = ['Normal', 'Normal', 'Normal', 'Normal', 'Bernoulli',
    ↪'Bernoulli']
df
```

```
[30]: Feature|Default Distribution Type      Mean      Variance
0  balance | Yes      Normal  1718.8534899175334  127084.50907211451
1  balance | No      Normal    806.4452220460979  206827.67007859284
2  income | Yes      Normal  32128.86080803881  184822064.01256827
3  income | No      Normal  33451.150293618564  177429727.32170913
4  student | Yes      Bernoulli  0.36548223350253806  0.2319049704965343
5  student | No      Bernoulli  0.28847148026882646  0.20525568534033856
```

The prior probability is 0.0328 as computed below.

```
[31]: # prior probability of default

prior = sum(y_train=='Yes')/len(y_train)
prior
```

```
[31]: 0.032833333333333333
```

(c)

Below is the prediction results by naive Bayes. Despite the violation of the assumption, the model seems to perform reasonably well.

```
[32]: from sklearn.metrics import confusion_matrix
from pandas import DataFrame
import numpy as np

def predict(X, dist, score=False):
    res = []
    for student, balance, income in X.values:
        log_pDefault = np.log(prior) \
            + np.log(dist['student | Yes'].pmf(student=='Yes')) \
            + np.log(dist['balance | Yes'].pdf(balance)) \
            + np.log(dist['income | Yes'].pdf(income))
        log_pNoDefault = np.log(1-prior) \
            + np.log(dist['student | No'].pmf(student=='Yes')) \
            + np.log(dist['balance | No'].pdf(balance)) \
            + np.log(dist['income | No'].pdf(income))

        predict_score = log_pDefault - log_pNoDefault
        res.append(predict_score if score else (predict_score > 0))

    return res

tn, fp, fn, tp = confusion_matrix(y_test=='Yes', predict(X_test, dist_norm)).
    ↪ravel()
print()
print('Misclassification Rate: ', (fn + fp)/(tn + fp + fn + tp))
print()
DataFrame([[tp, fp], [fn, tn]], columns=['True Y=1', 'True Y=0'], index=['Guess_
    ↪Y=1', 'Guess Y=0'])
```

Misclassification Rate: 0.02825

```
[32]:          True Y=1  True Y=0
Guess Y=1         36      13
Guess Y=0        100     3851
```

```
[33]: from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt

def plot_roc(dist, label):
    fig, ax = plt.subplots(1, 1, figsize=(5, 5))
    fpr, tpr, _ = roc_curve(y_test=='Yes', predict(X_test, dist, score=True))
    ax.plot(fpr, tpr, label=label)
    ax.plot([0, 1], [0, 1], 'k--')
    ax.set( aspect=1,
            xlabel='1-Specificity (False Positive Rate)',
```

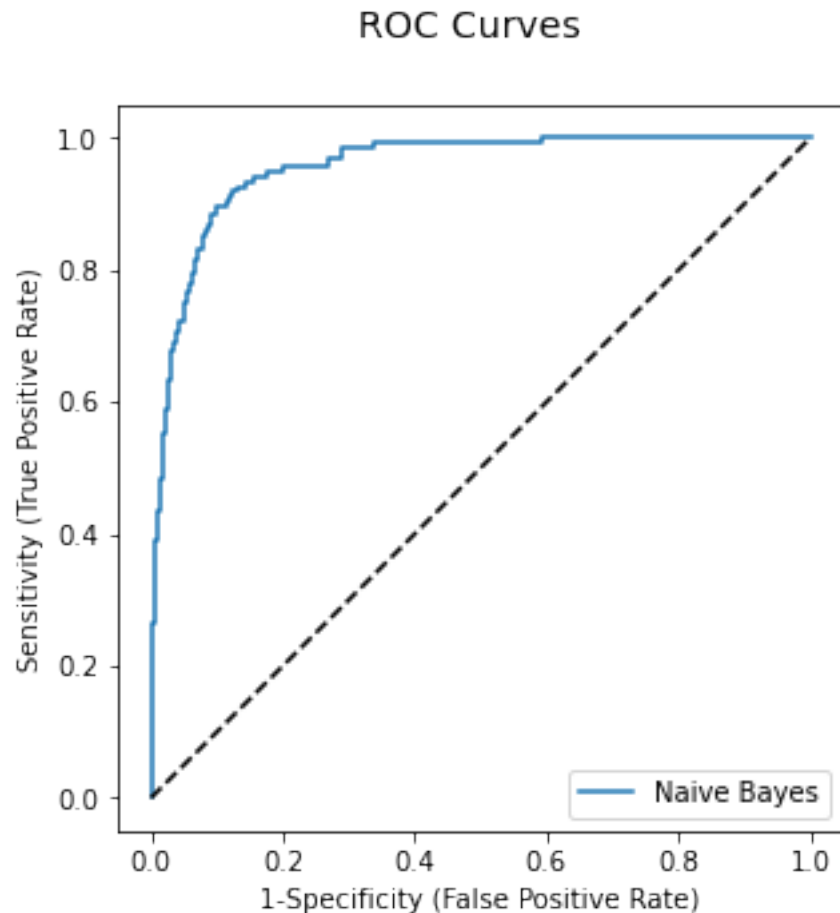
```

        ylabel='Sensitivity (True Positive Rate)')

    fig.suptitle('ROC Curves', fontsize=14)
    plt.legend()
    plt.show()

plot_roc(dist_norm, 'Naive Bayes')

```



(d)

I find the API of `scipy.stats.gaussian_kde` more convenient than `sklearn.neighbors.KernelDensity`, that's what I use for this part of the assignment. The below charts suggest that the default bandwidth of `scipy.stats.gaussian_kde` gives reasonable fit. We will stay with the default.

```

[34]: from pandas import DataFrame
      from scipy.stats import gaussian_kde

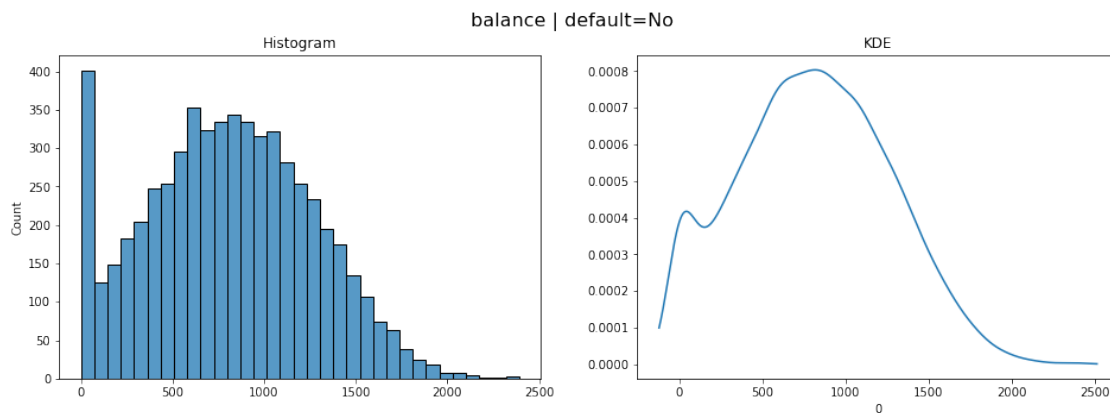
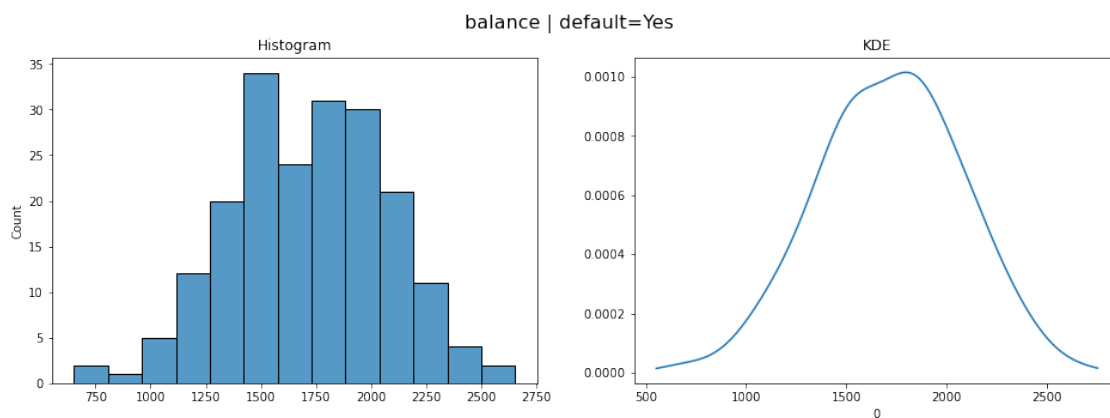
```

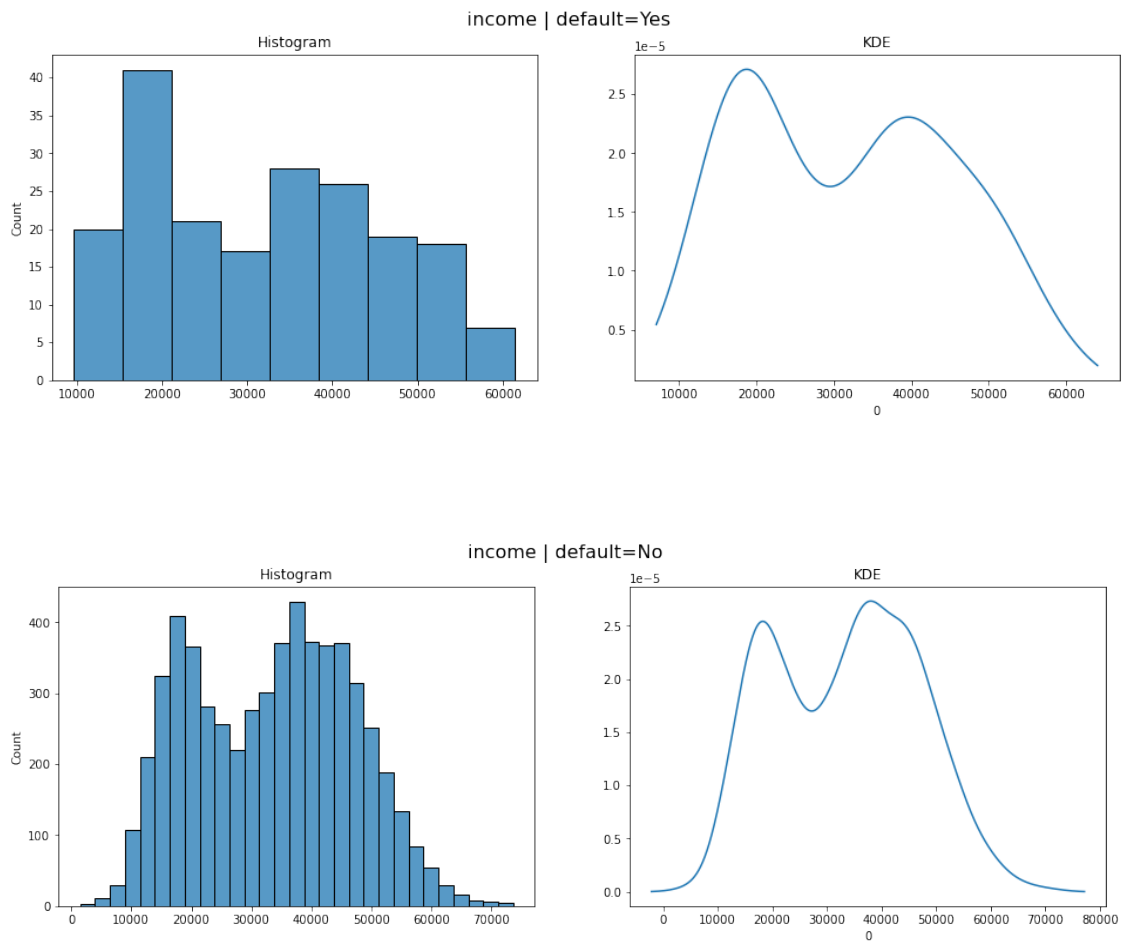
```

import seaborn as sns
import matplotlib.pyplot as plt
from math import floor, ceil

for feature in ['balance', 'income']:
    for default in ['Yes', 'No']:
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
        data = X_train[y_train==default][feature].values
        sns.histplot(data, ax=ax1)
        a, b = ax1.get_xlim()
        a, b = floor(a), ceil(b)
        DataFrame(zip(range(a, b), gaussian_kde(data).pdf(range(a, b)))).
        ↪set_index(0).plot(legend=None, ax=ax2)
        ax1.set(title='Histogram')
        ax2.set(title='KDE')
        plt.suptitle(feature + ' | default=' + default, fontsize=16)
        plt.show()

```





Like `scipy.stats.norm`, `scipy.stats.gaussian_kde` also provides a `pdf` method. Thus, after replacing the normal distribution by a `gaussian_kde` object, we can reuse the code in 2(b) to produce the same outputs. The results are very similar to the normal distribution results.

```
[35]: from scipy.stats import norm, bernoulli
      from pandas import DataFrame
      from scipy.stats import gaussian_kde

      dist_kde = dist_norm.copy()
      for feature in ['balance', 'income']:
          for default in ['Yes', 'No']:
              data = X_train[y_train==default][feature].values
              dist_kde[feature + ' | ' + default] = gaussian_kde(data)
```

```
[36]: from sklearn.metrics import confusion_matrix
      from pandas import DataFrame
```

```

tn, fp, fn, tp = confusion_matrix(y_test=='Yes', predict(X_test, dist_kde)).
    ↪ravel()
print()
print('Misclassification Rate: ', (fn + fp)/(tn + fp + fn + tp))
print()
DataFrame([[tp, fp], [fn, tn]], columns=['True Y=1', 'True Y=0'], index=['Guess_
    ↪Y=1', 'Guess Y=0'])

```

Misclassification Rate: 0.028

```

[36]:
      True Y=1  True Y=0
Guess Y=1      39      15
Guess Y=0     97    3849

```

```

[37]: from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 1, figsize=(5, 5))

fpr, tpr, _ = roc_curve(y_test=='Yes', predict(X_test, dist_norm, score=True))
ax.plot(fpr, tpr, label='Normal')

fpr, tpr, _ = roc_curve(y_test=='Yes', predict(X_test, dist_kde, score=True))
ax.plot(fpr, tpr, label='KDE')

ax.plot([0, 1], [0, 1], 'k--')
ax.set( aspect=1,
        xlabel='1-Specificity (False Positive Rate)',
        ylabel='Sensitivity (True Positive Rate)')

fig.suptitle('ROC Curves', fontsize=14)
plt.legend()
plt.show()

```

ROC Curves

